

LAB9_Report

402102562 - 402101758

AMITIS MIRABEDINI – NEGIN RAKI

Q1:

Using the script below, we obtain the required results:

```
Editor - E:\6th Semester\MISP Lab\MyLab\LAB9\Lab9.m
Lab9.m
+
3 %% Q1
4 img = imread('S3_Q1_utils/thorax_t1.jpg');
5 I = double(img(:,:,1));
6 [h, w] = size(I);
7
8
9 seed_lung = [90, 90]; % example lung center (row, col)
10 I_lung = I(seed_lung(1), seed_lung(2));
11
12 seed_lung2 = [90, 180]; % example lung center (row, col)
13 I_lung2 = I(seed_lung2(1), seed_lung2(2));
14
15 seed_liver = [140, 125]; % example liver center (row, col)
16 I_liver = I(seed_liver(1), seed_liver(2));
17
18 delta_lung = 15;
19 th_low_lung = I_lung - delta_lung;
20 th_high_lung = I_lung + delta_lung;
21
22 th_low_lung2 = I_lung2 - delta_lung;
23 th_high_lung2 = I_lung2 + delta_lung;
24
25 delta_liver = 20;
26 th_low_liver = I_liver - delta_liver;
27 th_high_liver = I_liver + delta_liver;
28
29 mask_lung = regionGrow(I, seed_lung, th_low_lung, th_high_lung);
30 mask_lung2 = regionGrow(I, seed_lung2, th_low_lung2, th_high_lung2);
31 mask_liver = regionGrow(I, seed_liver, th_low_liver, th_high_liver);
32
33 I_norm = mat2gray(I);
34 RGB = repmat(I_norm, [1 1 3]);
35
36 RGB_lung = RGB;
37 RGB_lung(:,:,2) = RGB_lung(:,:,2) + 0.6 * mask_lung + 0.6 * mask_lung2;
38 RGB_lung(RGB_lung > 1) = 1;
39
40 RGB_liver = RGB;
41 RGB_liver(:,:,2) = RGB_liver(:,:,2) + 0.6 * mask_liver;
42 RGB_liver(RGB_liver > 1) = 1;
43
44 figure;
45 imshow(RGB_lung);
46 title('Lung Region Segmentation');
47
48 figure;
49 imshow(RGB_liver);
50 title('Liver Region Segmentation');
51
52 %% Functions
53 function mask = regionGrow(I, seed, th_low, th_high)
54 [h, w] = size(I);
55 mask = false(h, w);
56 queue = seed;
57 head = 1;
58 while head <= size(queue,1)
59     x = queue(head,1);
60     y = queue(head,2);
61     head = head + 1;
62     % check bounds
63     if x < 1 || x > h || y < 1 || y > w, continue; end
64     if mask(x,y), continue; end
65     val = I(x,y);
66     if val >= th_low && val <= th_high
67         mask(x,y) = true;
68         % enqueue 4 neighbors
69         queue(end+1:end+4, :) = [ ...
70             x+1, y; ...
71             x-1, y; ...
72             x, y+1; ...
73             x, y-1 ...
74         ];
75     end
76 end
77 end
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
```

The process is as follows:

1. Image Loading and Preprocessing – The image is read and converted to grayscale (``I``). Pixel intensities are cast to double for numerical accuracy.

2. Seed Point Selection – Three seed points are manually defined:

- Two seeds for the left and right lungs (***seed_lung, seed_lung2***), placed within the lung cavities.

- One seed for the liver (***seed_liver***), placed within the liver tissue.

3. Threshold Calculation – For each seed, a local intensity range is established:

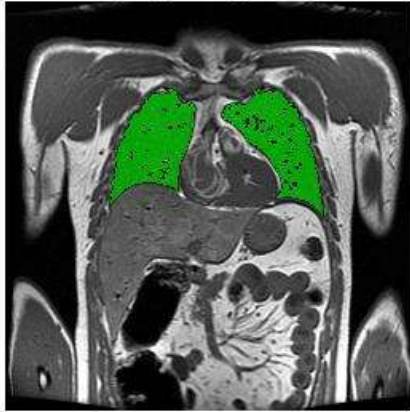
- Lungs: seed intensity ± 15 Hounsfield units (approximate CT range for air-filled tissue).

- Liver: seed intensity ± 20 Hounsfield units (slightly wider range to account for softer tissue variability)

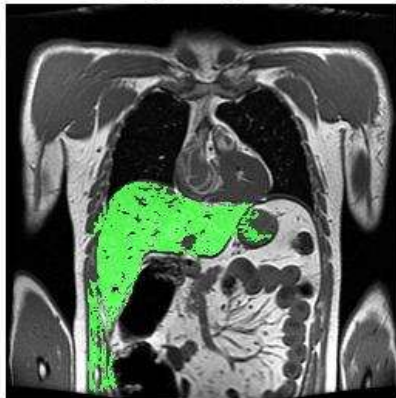
4. Region Growing – The custom `regionGrow` function is called for each seed. Starting from the seed, it expands to neighbouring pixels whose intensities fall within the defined lower/upper thresholds, generating a binary mask for each organ.

5. Visualization – A normalized grayscale version of the original image (``I_norm``) is replicated into three colour channels to create a base RGB image. The segmented masks are then overlaid in green by adding 0.6 to the green channel (lungs combined from both masks, liver separately). The result is clipped to $[0,1]$ and displayed in two figures, showing the lung and liver segmentations respectively.

Lung Region Segmentation



Liver Region Segmentation



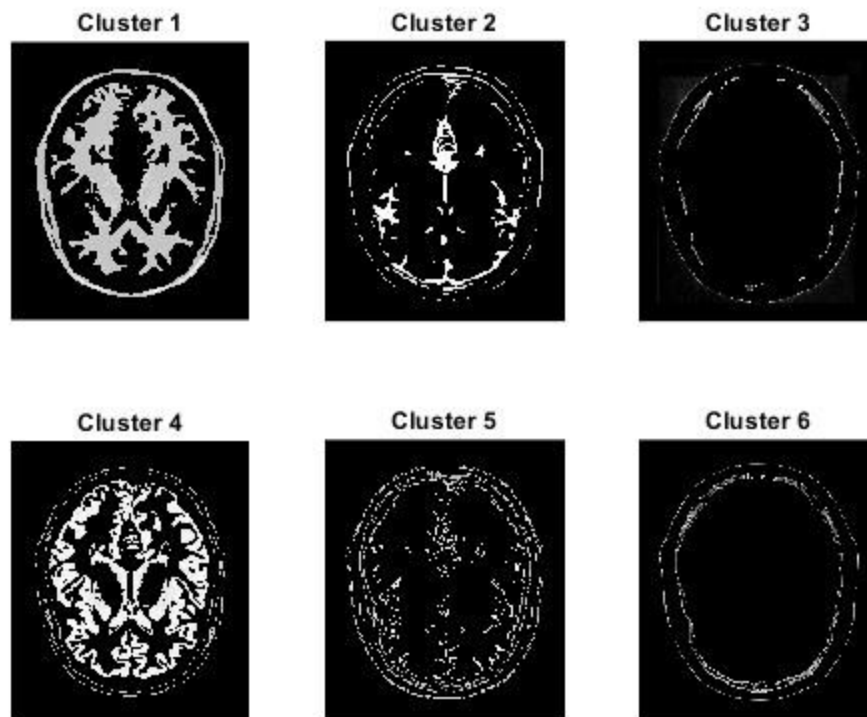
As we can see in the above figures, the segmentation was done successfully and the lung and liver regions are clearly identified.

Q2:

Using the MATLAB script below, we obtain the required results:

```
51 %% Q2
52 Img1 = imread("S3_Q2_utils\pd.jpg"); Img1 = Img1(:, :, 1);
53 Img2 = imread("S3_Q2_utils\t1.jpg"); Img2 = Img2(:, :, 1);
54 Img3 = imread("S3_Q2_utils\t2.jpg"); Img3 = Img3(:, :, 1);
55 volume = cat(3, Img1, Img2, Img3);
56
57 [m, n, ~] = size(volume);
58
59 data = double(reshape(volume, [], 3));
60
61 num_clusters = 6;
62 [labels, ~] = kmeans(data, num_clusters, 'Replicates', 5);
63
64 segmented_image = reshape(labels, m, n);
65
66 [m, n] = size(segmented_image);
67
68 slice1 = volume(:, :, 1); % Just once outside the loop
69 figure;
70 for k = 1:num_clusters
71     mask = (segmented_image == k);
72
73     cluster_img = zeros(m, n, 'uint8');
74     cluster_img(mask) = slice1(mask);
75
76     subplot(2,3,k);
77     imshow(cluster_img, []);
78     title(['Cluster ' num2str(k)]);
79 end
80
81
```

This script performs unsupervised tissue segmentation on a three-slice image volume using **k-means clustering**. Each spatial location is represented by a three-dimensional feature vector consisting of its intensity values across the three input slices. In this demonstration, only the first column of each image is used (making the data essentially a 1-D line), but the principle extends to full 2-D slices.



As we can see above, different elements of the image are clustered into different clusters. If we look, we can see the skull is present in clusters 3 and 6, while the brain tissues are more apparent in clusters 1, 2 and 4. In order to have a better analysis we need to gather information from all clusters. However, this segmentation allows us to focus on a particular cluster in a specific case if needed.

Q3:

Using the script below, we obtain the required results:

```
k = 6;
iters = 100;
tol = 0.0001;

% disp('data reshaped ok')
% size(data)

rng(1); % keep this so my results dont change on every run
idx = randperm(size(data,1), k);
cents = data(idx, :);

% disp('initial centers:')
% disp(cents)

for i = 1:iters

%   disp(['iter: ', num2str(i)])

% calc distances to all centers
dist = zeros(size(data,1), k);

    for j = 1:k
        % distance formula
        diff = data - cents(j, :);

        dist(:, j) = sqrt(sum(diff.^2, 2));
    end

% find closest one
[min_val, labels] = min(dist, [], 2);

    new_cents = zeros(k, 3);

    for j = 1:k
        pts = data(labels == j, :);

        % if the cluster isnt empty update it
        if size(pts, 1) > 0
            new_cents(j, :) = mean(pts, 1);
        else
            % disp('warning empty cluster')
```

```

|
for i = 1:iters

%   disp(['iter: ', num2str(i)])

% calc distances to all centers
dist = zeros(size(data,1), k);

for j = 1:k
    % distance formula
    diff = data - cents(j, :);

    dist(:, j) = sqrt(sum(diff.^2, 2));
end

% find closest one
[min_val, labels] = min(dist, [], 2);

new_cents = zeros(k, 3);

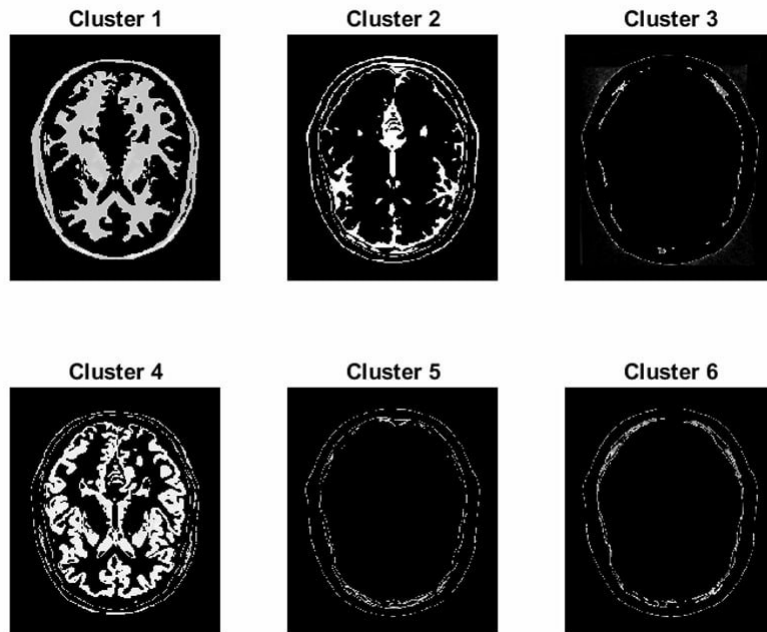
for j = 1:k
    pts = data(labels == j, :);

    % if the cluster isnt empty update it
    if size(pts, 1) > 0
        new_cents(j, :) = mean(pts, 1);
    else
        % disp('warning empty cluster')
        new_cents(j, :) = data(randi(size(data,1)), :);
    end
end
% err = sum(abs(new_cents - cents)) % old way I tried

err = max(vecnorm(new_cents - cents, 2, 2));

if err < tol
    disp(['converged at iter ' num2str(i)]);
    break;
end
cents = new_cents;
end

```

The only difference is that the clusters correspond to different parts and regions. For instance, brain tissues appear across three clusters, whereas the skull is represented by a separate set of clusters.

Q4:

Using the script below, we obtain the required results:

```
clc;
% assume volume is loaded in workspace
[r, c, d] = size(volume);

disp('dims are:')
disp(r)
disp(c)
% reshape so fcm works (needs 2d)
data = double(reshape(volume, r*c, 3));

disp('data size before fcm:')
disp(size(data))

k = 6; % number of clusters
[cents, U] = fcm(data, k, [2.0, 100, 1e-5, 0]);
disp('fcm finished!')
% find max for each pixel
[m_val, lbls] = max(U);

%disp(size(lbls))

% back to 2d image format
res = reshape(lbls, r, c);
s1 = volume(:,:,1); % just take the first slice

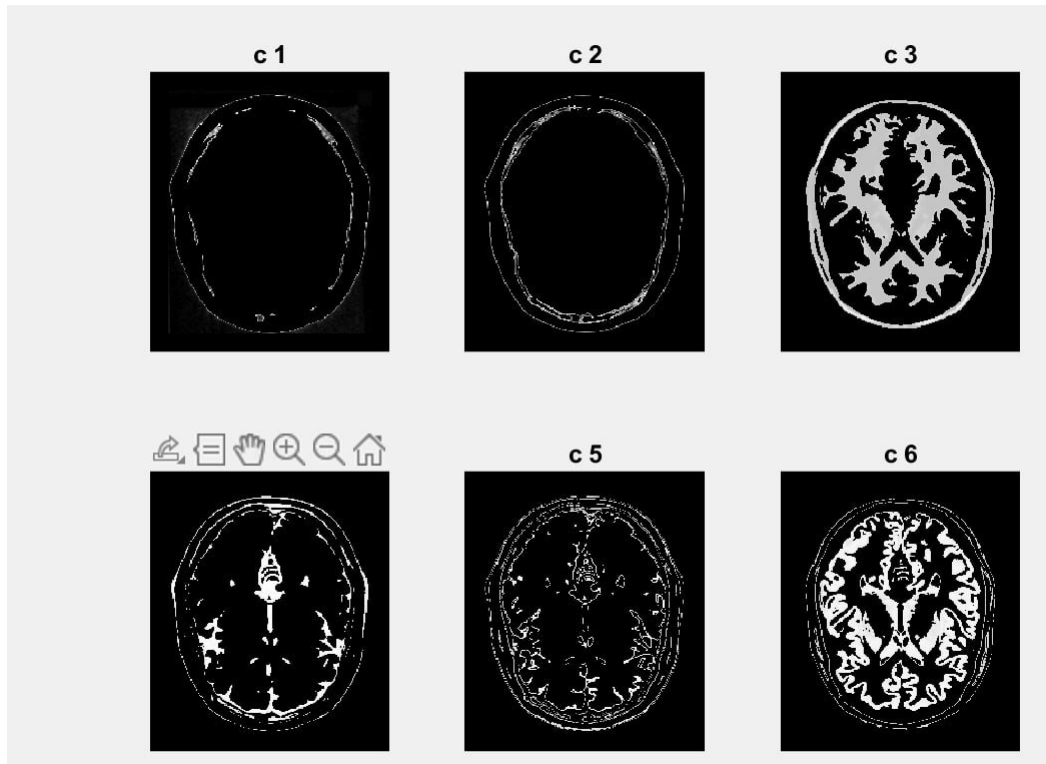
figure;
for i = 1:k
    m = (res == i);

    temp = zeros(r, c, 'uint8');

    temp(m) = s1(m); % put original pixels where mask is 1

    subplot(2,3,i);
    imshow(temp, []);
    title(['c ' num2str(i)]);
end

disp('done')
```



Although both clustering methods produce similar results, the main difference can be observed in how each method assigns clusters to different brain regions and tissues. The key differences between K-Means and Fuzzy C-Means (FCM) clustering are explained below:

K-Means Clustering

Hard clustering: Each pixel is assigned to only one specific cluster, meaning it has a single label.

Main goal: The algorithm tries to minimize the distance between pixels and their corresponding cluster centers, reducing the variation inside each cluster.

Cluster boundaries: It creates clear and sharp boundaries between different clusters because each pixel belongs to only one group.

Computational cost: It is usually faster and requires less computation compared with FCM.

Fuzzy C-Means (FCM) Clustering

Soft clustering: Instead of assigning a pixel to only one cluster, FCM gives each pixel a membership value for all clusters, ranging from 0 to 1.

Main goal: The algorithm minimizes a weighted distance function where the contribution of each pixel depends on its membership degree.

Cluster boundaries: It produces smoother transitions between clusters, which makes it more suitable for medical images where boundaries between tissues may not be clearly defined.

Computational cost: It is generally slower than K-Means because it requires additional

calculations to update membership values during each iteration.

In summary, K-Means is simpler and faster but produces more rigid segmentation, while FCM provides more flexible segmentation by considering uncertainty between different tissue regions, although it requires more computational time

Q5:

Region growing is a classical pixel-based image segmentation technique that groups pixels (or voxels) into homogeneous regions based on predefined **similarity criteria**—such as intensity, color, texture, or gradient magnitude. The algorithm starts from one or more **seed points** (manually or automatically selected) and **iteratively expands** to include neighbouring pixels that satisfy the chosen similarity constraint. This expansion continues until no adjacent pixel meets the inclusion condition, effectively producing connected and closed segmentation regions.

Due to its **conceptual simplicity**, **computational efficiency**, and strong ability to capture connected anatomical structures, region growing is widely adopted in **medical imaging** applications, including the segmentation of tumours, organs (e.g., lungs, liver), and brain tissues in MRI and CT scans.

Despite its strengths, standard region growing suffers from several inherent limitations: **sensitivity to initial seed placement**, reliance on **fixed global thresholds**, and **susceptibility to noise** and intensity inhomogeneity. To overcome these challenges and enhance segmentation accuracy, numerous improved approaches have been proposed in the literature. These can be categorized as follows:

1. Enhanced Seed Point Selection

Seed selection is arguably the most critical factor influencing the final segmentation outcome.

- **Automatic seed generation:** Instead of manual placement, automated strategies are employed to locate optimal seeds. For instance, **Harris corner detection** has been used to automatically identify robust seed points, improving both speed and reproducibility. Other works utilize **unsupervised learning** to cluster surface features and select local maxima of the cluster map as seeds, effectively eliminating user bias.
- **Multi-seed strategies:** Using multiple seeds (e.g., separate seeds for left and right lungs or for distinct tumour components) ensures comprehensive coverage of complex or multi-part structures, preventing under-segmentation.

- **Optimization-driven seeding:** Recent studies integrate meta-heuristic algorithms, such as the **Falcon Optimization Algorithm (FOA)**, to dynamically determine both the optimal seed positions and the associated similarity thresholds, leading to a fully adaptive segmentation pipeline.

2. Refinement of Similarity Criteria

Relying solely on a fixed intensity range often fails in images with overlapping intensity distributions or gradual intensity variations.

- **Adaptive thresholding:** Rather than using a global, user-defined delta, **Modular Adaptive Region Growing** employs image-specific, locally adaptive thresholds that adjust to the local statistics (mean and variance) of the growing region, making the process robust to spatial intensity drifts.
- **Multi-feature criteria:** Incorporating additional descriptors—such as **texture features** (e.g., Local Binary Patterns), **gradient magnitude**, and **directional features**—significantly improves differentiation between tissues with similar intensities but different textures.
- **Fuzzy logic integration:** The **Fuzzy Information Seeded Region Growing (FISRG)** method combines fuzzy membership functions with the growing process, allowing pixels to belong partially to multiple regions. This reduces the hard-cutoff effect and mitigates over-segmentation, particularly in ambiguous boundary areas.

3. Image Preprocessing

Improving the input image quality directly enhances the reliability of the growing process.

- **Noise reduction:** Applying smoothing filters (e.g., Gaussian, median, or anisotropic diffusion) suppresses noise and artefacts, promoting more homogeneous regions and preventing premature leakage.
- **Contrast enhancement:** Techniques like histogram equalization or contrast-limited adaptive histogram equalization (CLAHE) sharpen the boundaries between different tissues, making the similarity criterion more discriminative.
- **Edge enhancement:** Using edge-detection operators (e.g., Canny or Sobel) prior to segmentation can provide a guiding map that constrains growth and prevents the region from crossing true anatomical boundaries.

4. Postprocessing Refinements

The initial segmentation output often contains small holes, isolated islands, or ragged boundaries.

- **Morphological operations:** Applying **closing** fills small internal holes, while **opening** removes spurious pixel clusters attached to the main region.
- **Region merging:** Adjacent small fragments that share similar intensity statistics can be merged to form a single, coherent anatomical structure, reducing over-segmentation artefacts.

5. 3D Extension

For volumetric data (e.g., CT or MRI stacks), extending the connectivity criterion from 2D (8-neighbourhood) to **3D (26-neighbourhood)** ensures that the segmentation respects spatial continuity across slices. This prevents the loss of structural integrity in the z-direction and is essential for accurate organ volume measurement.

6. Hybrid Approaches

Integrating region growing with other advanced computational paradigms has yielded state-of-the-art results:

- **Combination with K-means clustering:** Using K-means as a preliminary step to initialize seeds and estimate intensity ranges significantly reduces manual intervention and improves convergence.
- **Integration with deep learning:** Convolutional Neural Networks (CNNs) or U-Net architectures can be trained to predict initial seed maps or probability masks, which then guide the region-growing process. This hybrid approach combines the global semantic understanding of deep networks with the local boundary-refining capability of region growing.
- **Fusion with optimization algorithms:** As mentioned earlier, meta-heuristic algorithms (e.g., FOA, Genetic Algorithms) can jointly optimize seed positions, thresholds, and even the similarity weighting coefficients, resulting in a fully automatic and robust system.

Q6:

1. Level-set Method

The Level-set method is a pretty advanced way to do image segmentation. Basically, it represents the edges of an object as a "zero-level contour" of a higher-dimensional function. The cool thing about this is that the curve can evolve over time using partial differential equations. It is really robust because it naturally handles topological changes—meaning if a shape splits into two, merges, or has holes in it, the algorithm just handles it without us needing to explicitly track those changes.

In medical imaging, this is used a lot for MRI, CT, and ultrasound images. It's really good at segmenting complex organs like the liver or heart, or finding tumors where the boundaries are super irregular and messy.

2. Snake Method (Active Contours)

The Snake method is basically a parametric curve that changes its shape to minimize its "energy". This energy is a balance of two things:

Internal forces: These try to keep the curve smooth and continuous so it doesn't get all jagged.

External forces: These pull the curve towards important features in the image, usually the edges.

Classic snakes rely heavily on finding strong edges, but there are also region-based versions that look at the overall image stats, making them better at handling noise. In medical imaging, we use active contours to find the exact boundaries of structures that have relatively clear edges, like brain ventricles in an MRI.

3. Explanation of the Snakes_demo code

I ran the demo code provided in the folder, and here is a brief breakdown of how the execution process works:

First, the code reads a grayscale brain image and sets up an initial contour (the snake). It does this by creating a circle in the center of the image. It uses numNodes to generate points in polar coordinates and then converts them to regular cartesian x_0 , y_0 coordinates.

It plots this starting circle on top of the brain image.

Then it calls the main Snake function to start shrinking the contour.

There are several parameters that control how the snake behaves, like α_0 , β_0 , ω , gradblur, δ_t , and timesteps.

To figure out the external energy, it blurs the image and calculates the gradient magnitude. The gradient basically acts as a force that pulls the snake towards the strong edges of the brain.

For the internal energy (which keeps the snake smooth), it builds a pentadiagonal matrix called A.

It precomputes the inverse of this matrix (γA^{-1}) before the main loop so the program runs faster in each iteration.

Inside the loop, the snake's x and y coordinates are updated step-by-step by combining the external image forces and the internal smoothness forces.

Every few iterations, it plots the snake so we can visually see it evolving and snapping to the brain boundary.

Finally, it shows the initial circle and the final result together so we can see the successful segmentation.